

Study Pilot: AI-Powered Study Schedule Generator

An end-to-end AI application project exploring unstructured data extraction and intelligent planning to evaluate course syllabi, identify priority topics, analyze exam proximity, and generate structured daily study schedules using Llama 3.1 and Streamlit.

Project Access & Resources

[!\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) Live Interactive Dashboard](#)[!\[\]\(e3f8612927870f2e0f9f5989e6dd3064_img.jpg\) Sample Data: sample.pdf](#)[!\[\]\(003082e50e3009141f59bd5df831749f_img.jpg\) Outputs: syllabus_output.json | timetable.json](#)

Problem Statement

Students frequently struggle to organize unstructured syllabus documents into actionable, balanced study plans. Key dates, chapter weightages, and course requirements are often fragmented across raw PDF files, forcing students to guess what to study and when.

This project solves these challenges by leveraging **Llama 3.1 via Groq API** to execute data extraction and dynamic schedule generation, paired with an interactive **Streamlit Dashboard** utilizing a 3-page navigation flow, Minimal Aurora UI, and automated PDF exports for student success.

Architecture & Workflow Phases

Phase 1: Data Ingestion & Extraction Analysis (extract.py)

To process raw course documents, the ingestion engine extracts unstructured text and utilizes AI to transform it into structured data formats.

- **Process:** Utilizes `pdfplumber` to read raw PDF text.
- **AI Integration:** Passes text to Llama 3.1 to identify subjects, chapters, and exam metadata.
- **Output Validation:** Generates `syllabus_output.json` to ensure data integrity before moving to the planning phase.

Phase 2: Priority Calculation & AI Planning (planner.py)

Analyzes subject weightage and exam proximity to identify peak priority topics and allocate study time effectively.

- **The "Brain":** Calculates subject priorities based on how close the exam is and the specific weightage of the chapters.
- **Schedule Generation:** Interfaces with Llama 3.1 via the Groq Cloud API to map out a highly structured daily study schedule.
- **Output Validation:** Generates `timetable.json` containing the final, actionable study schedule.

Phase 3: Automated Export Distribution (pdf_export.py)

This analysis breaks down the daily schedule into actionable chunks that the user can take offline or print.

- **Formatting:** Converts the generated JSON timetable into a clean, printable PDF document using Python formatting libraries.
- **Accessibility:** Ensures the study schedule is available directly through the Streamlit interface for immediate download.

Phase 4: User Engagement & Notifications (reminder.py)

This phase evaluates user engagement by managing the logic required to keep the student on track with their generated goals.

- **Automated Triggers:** Manages the logic for sending daily email notifications.
- **Accountability:** Acts as a push-notification system reflecting the daily tasks outlined in `timetable.json`.

Strategic Recommendations & Action Plan

Based on the AI planning logic and data extraction capabilities, the following core study strategies are automated by the system:

Study Shift Optimization:

- **Peak Priority Focus:** High-weightage subjects and imminent exams are scheduled with significantly scaled-up study blocks.
- **Daily Pacing:** Concentrate maximum mental resources on complex chapters early in the schedule to prevent cramming.

Material & Inventory Allocation:

- Prioritize tracking for core subjects outlined in the `syllabus_output.json` file.
- Maintain consistent adherence to the daily milestones generated in the `timetable.json` to ensure all topics are covered before exam day.

Technical Tool Stack & Development Workflow

- **Python & Llama 3.1 (Groq API):** Applied for intelligent text parsing, JSON data structuring, and logical schedule generation—utilizing advanced prompting logic to execute multi-layered calculations based on exam dates and chapter weightages.
- **Streamlit (App.py & Style.css):** Leveraged for the main entry point, managing user interactions, 3-page navigation flow, and dynamic UI rendering (Minimal Aurora styles).
- **pdfplumber:** Utilized for high-fidelity text extraction from unstructured raw PDF syllabi.
- **Data Verification:** Cross-validated raw PDF text extraction against `syllabus_output.json` and `timetable.json` structures to ensure data accuracy and consistency across both the extraction and planning environments.

Conclusion

This project demonstrates an end-to-end AI analytics workflow, turning raw syllabus documents into clear, student-focused insights and schedules.

By combining pdfplumber data extraction with the dynamic reasoning capabilities of Llama 3.1 via the Groq API, the project provides a structured view of academic requirements—highlighting key exam dates, priority weightages, and daily study distributions. The findings offer a strong foundation for supporting data-driven decisions in student time management and exam preparation.

Future Work

- **Interactive Calendar Deployment:** Replicating the data model to sync directly with Google Calendar or Apple Calendar to enable automated visual updating on mobile devices.
- **Expanded Trend Analysis:** Incorporating longer-term historical data from user study habits to adjust the AI's scheduling algorithm dynamically.
- **Predictive Analytics:** Exploring forecasting models to estimate the time required per chapter based on the user's past completion rates, optimizing future schedule generation.